

1.

Code:

```
# Remove unnecessary columns
data = data[['v1', 'v2']]
data.columns = ['label', 'message']

# Calculate class probabilities
class_probabilities = data['label'].value_counts(normalize=True)

print("Class Probabilities:")
print(class_probabilities)

# Features and target
X = data['message']
y = data['label']

# Convert text into numerical features
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Train Multinomial Naive Bayes model
model = MultinomialNB()
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Predict a new message
new_message = ["Congratulations! You have won a free lottery. Claim your prize now."]
new_message_vector = vectorizer.transform(new_message)

prediction = model.predict(new_message_vector)
posterior_probability = model.predict_proba(new_message_vector)

print("\nPredicted Class:", prediction[0])
print("Posterior Probabilities:", posterior_probability)

# Evaluate model
print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Output:

```
Class Probabilities:
label
ham      0.865937
spam     0.134063
Name: proportion, dtype: float64

Predicted Class: spam
Posterior Probabilities: [[1.01185116e-07 9.99998999e-01]]

Accuracy: 0.97847533632287

Confusion Matrix:
[[952  13]
 [ 11 139]]

Classification Report:
              precision    recall  f1-score   support

     ham      0.99      0.99      0.99         965
     spam      0.91      0.93      0.92         150

   accuracy              0.98         1115
  macro avg      0.95      0.96      0.95         1115
 weighted avg      0.98      0.98      0.98         1115
```

2.

Code:

```
# Display first 5 rows
print("First 5 Rows of Dataset:")
print(data.head())

# Calculate class probabilities
class_probabilities = data['label'].value_counts(normalize=True)

print("\nClass Probabilities:")
print(class_probabilities)

# Features and target
X = data['message']
y = data['label']

# Convert text into numerical features
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Train model
model = MultinomialNB()
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# New SMS message
new_message = ["Congratulations! You have won a free lottery. Claim your prize now."]
new_message_vector = vectorizer.transform(new_message)

prediction = model.predict(new_message_vector)
posterior_probability = model.predict_proba(new_message_vector)

print("\nPredicted Class:", prediction[0])
print("\nPosterior Probabilities:")
print(posterior_probability)

# Model evaluation
print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Output:

```
First 5 Rows of Dataset:
  label message
0  ham  Go until jurong point, crazy.. Available only ...
1  ham           Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3  ham  U dun say so early hor... U c already then say...
4  ham  Nah I don't think he goes to usf, he lives aro...

Class Probabilities:
label
ham      0.865937
spam     0.134063
Name: proportion, dtype: float64

Predicted Class: spam

Posterior Probabilities:
[[1.01185116e-07  9.99998999e-01]]

Accuracy:  0.97847533632287

Confusion Matrix:
[[952  13]
 [ 11 139]]

Classification Report:
              precision    recall  f1-score   support

     ham       0.99       0.99       0.99         965
     spam       0.91       0.93       0.92         150

   accuracy              0.98         1115
  macro avg       0.95       0.96       0.95         1115
 weighted avg       0.98       0.98       0.98         1115
```

3.

Code:

```
import pandas as pd
import math

# Load dataset
data = pd.read_csv(r"C:\Users\umasi\Downloads\archive (12)\play_tennis.csv")

print("First 5 Rows of Dataset:")
print(data.head())

# Entropy function
def entropy(target):
    values = target.value_counts()
    total = len(target)
    ent = 0
    for count in values:
        p = count / total
        ent -= p * math.log2(p)
    return ent

# Information Gain function
def information_gain(data, attribute, target):
    total_entropy = entropy(data[target])

    values = data[attribute].unique()
    weighted_entropy = 0

    for value in values:
        subset = data[data[attribute] == value]
        weighted_entropy += (len(subset) / len(data)) * entropy(subset[target])

    return total_entropy - weighted_entropy

# Target column
target = "play"

print("\nEntropy of Dataset:", entropy(data[target]))

print("\nInformation Gain of Attributes:")

gain_dict = {}

for column in data.columns:
    if column != target:
        gain = information_gain(data, column, target)
        gain_dict[column] = gain
        print(f"{column}: {gain:.4f}")

best_attribute = max(gain_dict, key=gain_dict.get)

print("\nBest Attribute (Highest Information Gain):", best_attribute)
```

Output:

```
First 5 Rows of Dataset:
  day outlook temp humidity wind play
0  D1   Sunny  Hot     High   Weak  No
1  D2   Sunny  Hot     High  Strong  No
2  D3  Overcast  Hot     High   Weak  Yes
3  D4    Rain  Mild    High   Weak  Yes
4  D5    Rain  Cool   Normal  Weak  Yes

Entropy of Dataset: 0.9402859586706311

Information Gain of Attributes:
day: 0.9403
outlook: 0.2467
temp: 0.0292
humidity: 0.1518
wind: 0.0481

Best Attribute (Highest Information Gain): day
```